

Testes Automatizados & Maestro

Por que testes importam?

Software sem testes não é software confiável.
É software com sorte.

A falácia dos "testes atrasam"

Testes custam tempo. Bugs em produção
custam reputação, dinheiro e clientes.

Trade-offs dos testes

O que você ganha	O que você perde
Refatoração segura Código novo ou mudanças drásticas podem ser feitas sem medo de consequência, aumentando a velocidade da equipe.	Tempo de desenvolvimento no curto prazo testes adicionam tempo de desenvolvimento.
Documentação viva Um teste bem escrito descreve o comportamento esperado do sistema, documentando casos comuns e de ponta do sistema.	Manutenção contínua Testes também quebram quando o sistema muda, sistemas que tomam formas diferentes causam atritos.
Confiança Casos de ponta não explorados manualmente podem ser facilmente testados e validados.	Falsos positivos Testes podem causar falsos positivos, movimentando esforço onde algo está funcionando.

QA vs. Tester

QA:

Garante que o processo de desenvolvimento minimize a chance de problemas aparecerem.

Tester:

Executa os testes e valida se o sistema se comporta como esperado.

O que realmente é um teste?

"Teste é o processo de avaliar um componente ou sistema para determinar se ele satisfaz os requisitos especificados e para identificar diferenças entre os resultados esperados e os resultados reais."

- ISTQB

Criando um cenário de teste

Um bom cenário não descreve o que você faz. Descreve o que o sistema deve garantir.

✗ "Clicar no botão deve funcionar"

✓ "Ao clicar em Confirmar Pedido, o pedido deve ser registrado e a tela de confirmação deve exibir o número do protocolo"

Gherkin

Uma linguagem simples para descrever o comportamento do sistema em formato que qualquer pessoa do time entende.

Estrutura

Given — o contexto inicial (dado que...)

Dado que o usuário está na tela de pedidos

When — a ação executada (quando...)

Quando ele clica em "Confirmar Pedido"
Então o pedido deve ser registrado no sistema

Then — o resultado esperado (então...)

E uma tela de sucesso deve exibir o número do protocolo

Testes bons VS ruins.

Um teste ruim dá falsa segurança. Às vezes é pior do que não ter nenhum.

```
function calcularFrete(valorDaCompra) {  
  if (valorDaCompra >= 200) {  
    return 0; // Frete grátis  
  }  
  return 20; // Frete fixo  
}
```

O código:

```
it('teste de frete', () => {  
  let frete1 = calcularFrete(250);  
  let frete2 = calcularFrete(50);  
  
  expect(frete1).toBeDefined();  
  expect(frete2).toBe(20);  
  expect(calcularFrete(200)).not.toBeNull();  
});
```

O teste ruim.

```
// Teste Bom 1: Acima do valor mínimo
it('deve retornar frete grátis (0) quando o valor da compra for maior que 200', () => {
  const valorDaCompra = 250;
  const resultado = calcularFrete(valorDaCompra);

  expect(resultado).toBe(0);
});

// Teste Bom 2: Abaixo do valor mínimo
it('deve retornar frete fixo de 20 quando o valor da compra for menor que 200', () => {
  const valorDaCompra = 150;
  const resultado = calcularFrete(valorDaCompra);

  expect(resultado).toBe(20);
});
```

Os testes bons.

Abordagens de testes

O que você pode ver no código define o quanto você consegue testar com precisão.

	Conhecimento do código	Quem geralmente usa	O que testa	Exemplo prático	Ferramenta típica
Whitebox	Total	Desenvolvedor	Lógica interna, caminhos, cobertura	Testar uma função de cálculo de juros	JUnit, Jest, NUnit
Graybox	Parcial	Dev ou QA	Integrações, fluxos críticos	Testar a integração entre o formulário e o banco	Postman, REST Assured
Blackbox	Nenhum	Qa ou usuário	Comportamento e interface	Testar se o botão de confirmar pedido funciona	Maestro, Cypress, Selenium

— Testes funcionais

O sistema faz o que foi prometido?

Essa é a pergunta dos testes funcionais.

- Testes unitários
- Testes de integração
- Testes E2E
- Testes regressivos
- Testes de smoke (Fumaça)

```
it('deve barrar um e-mail que não possui o símbolo @', () => {  
  
  // Ação e Validação: Passamos um texto incorreto e esperamos o erro  
  expect(() => {  
    validarEmail('joaogmail.com');  
  }).toThrow('E-mail inválido. Falta o arroba.');
```

Teste unitário

```

// Peça 1
class ValidadorDeEstoque {
  temNoEstoque(produto) {
    return produto === 'teclado'; // Simplificado para o exemplo
  }
}

// Peça 2
class CalculadoraDeDesconto {
  aplicar(valor) {
    return valor - 10;
  }
}

// Peça 3 (O Maestro que integra as outras duas)
class ServicoDeVendas {
  constructor(estoque, calculadora) {
    this.estoque = estoque;
    this.calculadora = calculadora;
  }

  realizarVenda(produto, valor) {
    // Inter-relação 1: Conversa com o estoque
    if (!this.estoque.temNoEstoque(produto)) {
      throw new Error('Produto sem estoque');
    }
    // Inter-relação 2: Conversa com a calculadora
    return this.calculadora.aplicar(valor);
  }
}

```

Teste integração

```
it('deve integrar o Estoque e a Calculadora com sucesso para realizar uma venda', () => {  
  
  // 1. Preparação: Pegamos as peças REAIS do nosso sistema  
  const validadorReal = new ValidadorDeEstoque();  
  const calculadoraReal = new CalculadoraDeDesconto();  
  
  // ENCAIXE: Passamos as peças para dentro do Serviço de Vendas  
  const servicoDeVendas = new ServicoDeVendas(validadorReal, calculadoraReal);  
  
  // 2. Ação: Chamamos a função principal  
  const valorFinal = servicoDeVendas.realizarVenda('teclado', 100);  
  
  // 3. Validação: A comunicação entre as três classes funcionou?  
  expect(valorFinal).toBe(90);  
  
});
```

Teste de integração

Testes não funcionais.

O sistema funciona. Mas aguenta? É rápido? É seguro? É acessível?

Cenários:

- Performance
- Acessibilidade
- Segurança

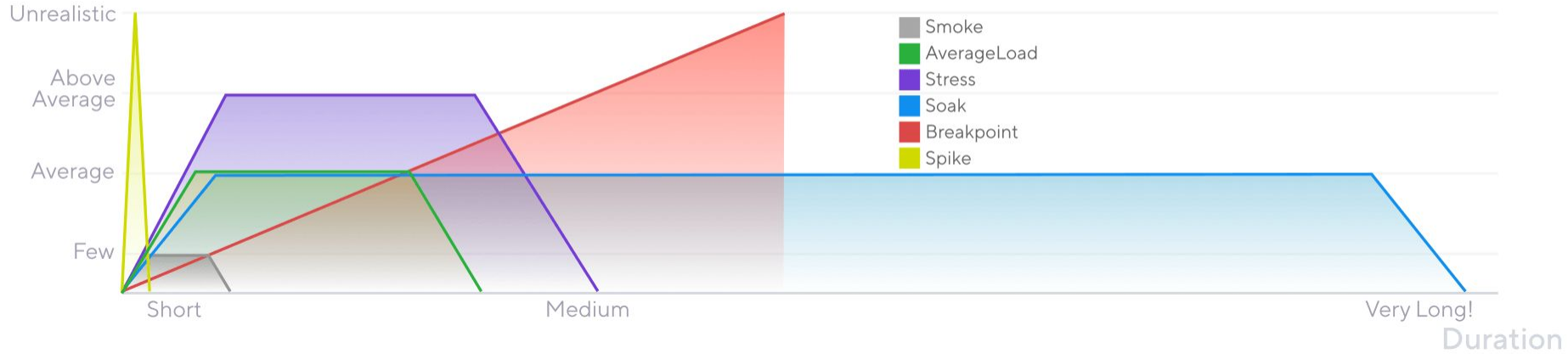


Testes de performance, carga, e estresse.

Performance testa o normal. Carga testa o limite.
Estresse testa o que acontece além dele.

Load test types at a glance

VUs/Throughput



**Tipos de teste
de carga**

PROJECTS

All pawel's tests

Public tests

MANAGE

🕒 Scheduled tests

EXPLORE

+ Integration

📄 Docs

🔍 CLI

🔗 Support

🟢 Finished ⌚ 20min 👤 50 VUs 🌐 Region: Ashburn 👤 Started by: pawel@loadimpact.com

COMPARE RESULT

Select a compare target ▾

PERFORMANCE OVERVIEW

REQUESTS MADE

104.2K reqs

HTTP FAILURES

4 reqs

PEAK RPS

100 reqs/s

AVG RESPONSE TIME

4 ms

PERFORMANCE INSIGHTS

Prithms have analyzed the test results and have not found any performance issues.

The response time of the system being tested was **4ms**, and **104.2K** requests were made with **4** failures, at an average request rate of **87**

Testes de performance

Testes de segurança

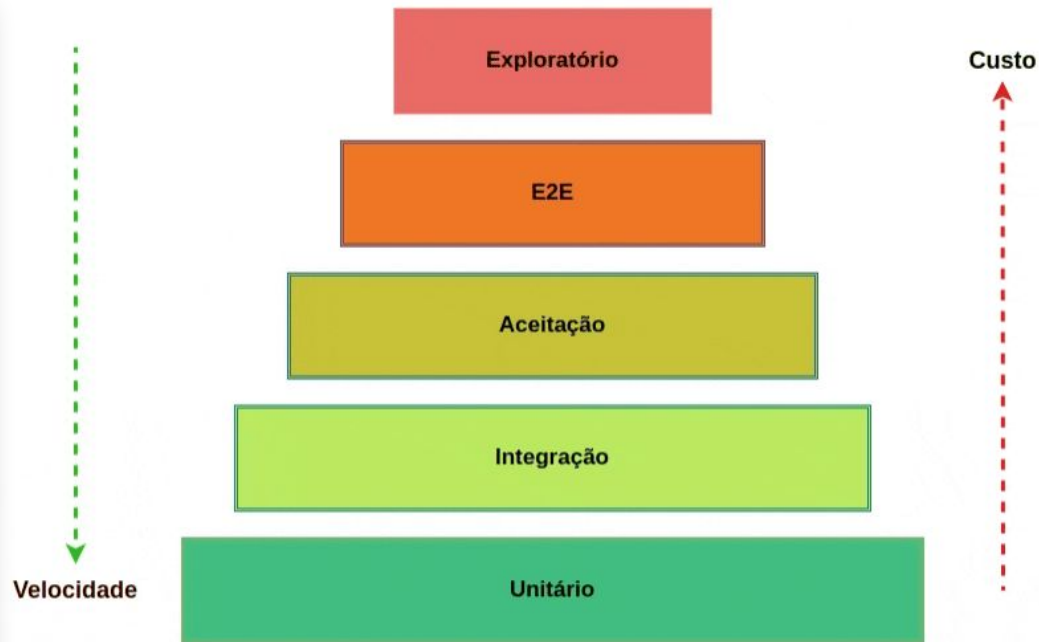
Sistemas públicos são alvos. OWASP Top 10 é o mínimo que toda equipe deveria conhecer.

Testes de acessibilidade.

No Brasil, acessibilidade digital não é opcional. É lei.

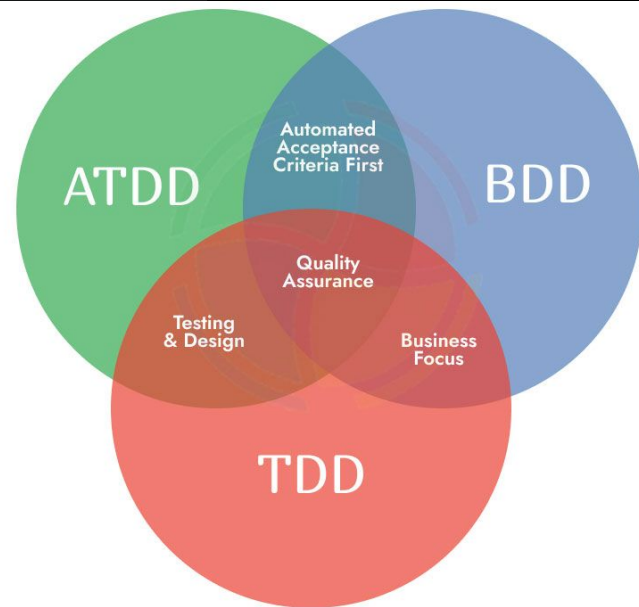
Pirâmide de testes

Quanto mais alto na pirâmide, mais lento, mais caro e mais frágil. Construa de baixo para cima.



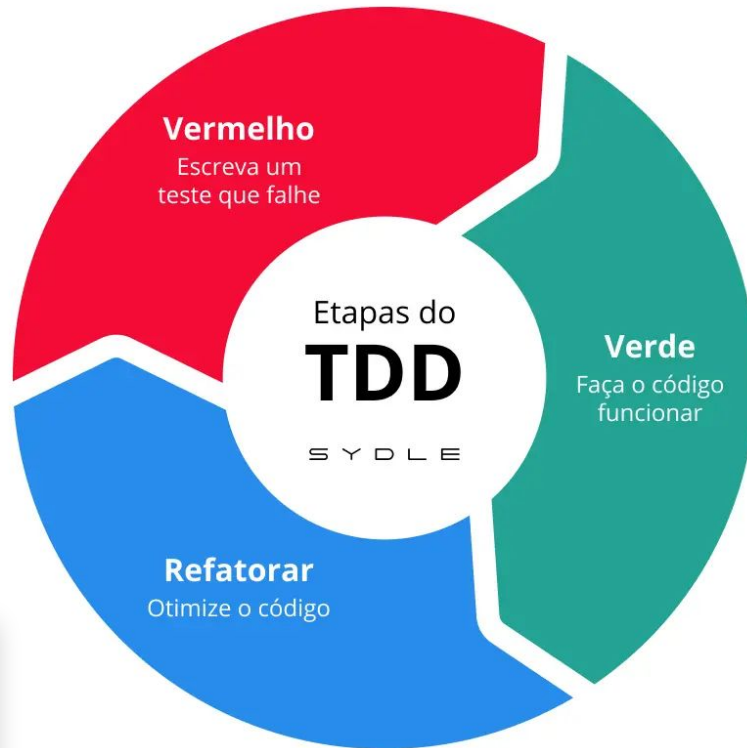
Metodologia de testes

TDD, BDD e ATDD não são sobre testes.
São sobre como uma equipe pensa antes de codar.



TDD

Primeiro você especifica o resultado esperado, depois você codifica.



Cenário: Cliente aplica cupom válido em compra acima do valor mínimo.
Dado que o cliente tem um carrinho com o valor total de R\$ 150,00
E possui um cupom "BEMVINDO10" válido
Quando o cliente insere o cupom "BEMVINDO10" no checkout
Então o valor total do carrinho deve ser atualizado para R\$ 135,00
E uma mensagem "Cupom aplicado com sucesso" deve ser exibida

BDD

Conectando múltiplas áreas de
negócio e técnica.

QA no fluxo de entrega

QA não é a última porta antes da produção.
É uma responsabilidade distribuída no time.

Maestro

Test Anything →

Próximos passos

Botando a mão na massa com
Maestro, ferramenta de testes de E2E.